Universidad Tecnológica de Chihuahua

Tecnologías de la Información área: Desarrollo y Gestión de Software



Desarrollo Web Integral Unidad 3

Implementación de Composite API y Consumo de Microservicios

Alumno:

Edwin Arturo Ledezma Rivas

Docente:

Dr. Alfonso Jose Barroso Barajas

Grupo: pendiente de completar

jueves, 29 de octubre de 2026

1 Introducción y caso de estudio

Esta práctica implementa una pantalla de detalle de producto que recibe catálogo, inventario y conversión USD a MXN mediante una única solicitud al servidor. El resultado es una aplicación de tres procesos ASP.NET Core independientes, un dashboard web y un contrato JSON consolidado. Las 26 verificaciones locales terminaron sin fallos y la consulta real a Frankfurter produjo una respuesta SUCCESS.

El problema de las múltiples peticiones

En una aplicación de comercio electrónico, el cliente necesita información que pertenece a distintos servicios. Si consulta directamente producto, almacén y divisas, debe administrar tres conexiones, tres contratos y sus errores. Esta comunicación repetitiva se conoce como Chatty I/O. En redes móviles, cada conexión agrega trabajo y puede incrementar la espera percibida.

La solución con una API agregadora

El patrón Composite API concentra la coordinación en el gateway: recibe el identificador, inicia las tres consultas asíncronas y transforma sus respuestas en un contrato preparado para la interfaz. Así el cliente realiza una petición para obtener el detalle, aunque el gateway mantiene tres consultas salientes. Este uso corresponde al patrón Gateway Aggregation descrito por Microsoft (s. f.-a).

```
Navegador
   | GET /api/v1/products/P-101/summary
   v
Gateway.Api :5100
   |---- HTTP ----> Catalog.Api :5101
   |---- HTTP ----> Inventory.Api :5102
   |---- HTTPS ---> Frankfurter externo
   v
JSON consolidado  -->  Dashboard del mismo origen
```

La concurrencia permite que la duración de red se aproxime a la dependencia más lenta, más el costo de agregación. No se afirma una aceleración fija: no se realizó una prueba de carga ni una comparación estadística de latencias. La mejora verificable en este alcance es una sola consulta de detalle desde el cliente y una orquestación concurrente comprobada.

Alcance de la entrega

El catálogo y el inventario contienen tres productos de demostración en memoria. La API externa es real durante la ejecución normal; únicamente las pruebas reproducibles sustituyen divisas por un servidor local controlado. No se implementan compras, pagos ni modificaciones de catálogo, porque el objetivo es integrar y consultar información.

2 Especificación técnica de las APIs

Todos los servicios exponen HTTP REST y JSON. Los identificadores tienen el formato P-101. Los servicios propios pueden ejecutarse y consultarse por separado; el gateway no comparte sus repositorios ni referencia sus ensamblados.

API	Tipo	Formato	Endpoint	Función
Catálogo	Propia	REST JSON	GET :5101/microservices/products/{id}	Información del producto y precio base USD.
Inventario	Propia	REST JSON	GET :5102/microservices/inventory/{id}	Unidades, almacén y plazo de entrega.
Frankfurter	Terceros	REST JSON HTTPS	GET api.frankfurter.dev /v2/rate/USD/MXN	Tasa diaria de referencia USD/MXN.
Composite	Propia	REST JSON	GET :5100/api/v1/products/{id}/summary	Consolida las tres respuestas.

Entradas y contratos individuales

Catálogo recibe id en la ruta y devuelve id, name, description, category y priceUsd. Inventario recibe el mismo identificador y devuelve productId, availableUnits, warehouse y estimatedDeliveryDays. Un ID inválido produce 400 y un producto no registrado, 404. Ambos ofrecen GET /health.

Frankfurter v2 responde con date, base, quote y rate. La consulta fija USD como base y MXN como destino; no necesita clave. Según su documentación oficial, las tasas son diarias y de referencia. La fecha publicada se conserva en exchange_rate_date para que la interfaz no las presente como una cotización intradía (Frankfurter, s. f.).

Contrato de la API compuesta

La salida utiliza snake_case y conserva los campos pedidos en la práctica: product_id, title, price_usd, price_mxn, stock_status, available_units, currency_conversion_source y status. Agrega descripción, categoría, almacén, plazo, tasa, fecha y observaciones de cada dependencia. Los campos ausentes por fallo valen null. El alias /api/v1/product-summary/{id} invoca el mismo caso de uso.

Productos para verificar el comportamiento

ID	Producto	USD	Unidades
P-101	Laptop Gamer XYZ	1200.00	15
P-102	Audífonos Studio	85.50	4
P-103	Teclado Mecánico Pro	59.99	0

3 Código y estrategia de orquestación

La solución usa Clean Architecture en cada servicio: Domain contiene modelos e invariantes; Application declara casos de uso y puertos; Infrastructure implementa repositorios o transporte HTTP; Api configura DI, rutas y presentación. El gateway depende de sus propias interfaces y de contratos HTTP, de modo que los servicios propios conservan independencia.

Concurrencia con Task WhenAll

ProductSummaryService inicia las tres operaciones antes de esperar sus resultados. Observe mide cada consulta y captura únicamente DependencyException. Una cancelación de la petición o un error inesperado se propaga al host.

```
var productTask = Observe("catalog",
    () => products.GetAsync(id, ct));
var stockTask = Observe("inventory",
    () => inventory.GetAsync(id, ct));
var rateTask = Observe("currency",
    () => rates.GetAsync(ct));
await Task.WhenAll(productTask, stockTask, rateTask);
```

Cliente HTTP y API externa

La raíz de composición registra clientes tipados con AddHttpClient. Configure establece BaseAddress, cabecera Accept y un timeout configurable de cuatro segundos. El adaptador usa un token enlazado para limitar también la lectura del cuerpo de la respuesta.

```
builder.Services
    .AddHttpClient<IExchangeRateClient, FrankfurterHttpClient>(
        h => Configure(h, "CurrencyUrl",
            "https://api.frankfurter.dev/"));


// Dentro de FrankfurterHttpClient.GetAsync
using var deadline =
    CancellationTokenSource.CreateLinkedTokenSource(ct);
deadline.CancelAfter(http.Timeout);
r = await JsonTransport.GetAsync<ExchangeRate>(
    http, "v2/rate/USD/MXN", "currency", deadline.Token);
```

Transformación de la respuesta

Se valida la pareja USD/MXN, una tasa positiva dentro del límite admitido y una fecha válida. price_mxn se calcula con decimal y redondeo de dos decimales AwayFromZero. Los datos recibidos del catálogo e inventario también se validan, incluido el identificador; los parámetros obligatorios de sus constructores JSON deben estar presentes.

```
return decimal.Round(priceUsd * rate, 2,
    MidpointRounding.AwayFromZero);
```

4 Tolerancia a fallos y buenas prácticas

Los adaptadores traducen errores HTTP, red, timeout y JSON inválido a DependencyException. La aplicación transforma los fallos opcionales en campos null y status PARTIAL. El controlador solo traduce el fallo crítico del catálogo a HTTP; no contiene lógica de conversión ni accede a repositorios.

Situación	HTTP	Respuesta
Tres fuentes disponibles	200	SUCCESS; producto, stock y conversión.
Inventario o divisas no disponibles	200	PARTIAL; campos afectados null.
ID mal formado	400	Problem Details sin trazas.
Producto no encontrado	404	Problem Details con NOT_FOUND.
Error crítico de catálogo	502	Catálogo no disponible.
Timeout de catálogo	504	Catálogo no disponible.

Control de excepciones

```
try
{
    return new(await action(), null,
        new(service, "SUCCESS", timer.ElapsedMilliseconds, null));
}
catch (DependencyException ex)
{
    return new(null, ex,
        new(service, "UNAVAILABLE",
            timer.ElapsedMilliseconds, ex.Code));
}
```

No se usa una tasa fija de respaldo ni se interpreta una caída de inventario como cero existencias. UNKNOWN significa que el stock no se pudo confirmar; OUT_OF_STOCK representa una consulta válida con cero unidades. Si no hay stock confirmado, estimated_delivery_days es null. El usuario puede repetir la consulta desde el dashboard.

Aplicación de SOLID y seguridad

Las responsabilidades de consulta, conversión, transporte y HTTP están separadas. Los tres puertos de cliente tienen una sola operación y pueden sustituirse mediante DI, como demuestran los dobles de prueba. Los casos de uso no construyen HttpClient ni conocen URLs. Esta dirección de dependencias sigue el enfoque de arquitectura limpia documentado por Microsoft (s. f.-b).

El frontend utiliza textContent y el gateway aplica CSP del mismo origen. Las URLs se configuran en el servidor y Frankfurter usa HTTPS. No se implementan autenticación, persistencia durable, caché ni reintentos automáticos; su necesidad deberá evaluarse antes de un despliegue público.

5 Evidencias de funcionamiento

Las figuras siguientes representan respuestas HTTP reales obtenidas el 29 de octubre de 2026 con un cliente programático. Los archivos JSON originales conservan URL, método, estado, fecha UTC, duración y cuerpo. Son registros de consultas; no son capturas de Postman ni del navegador.

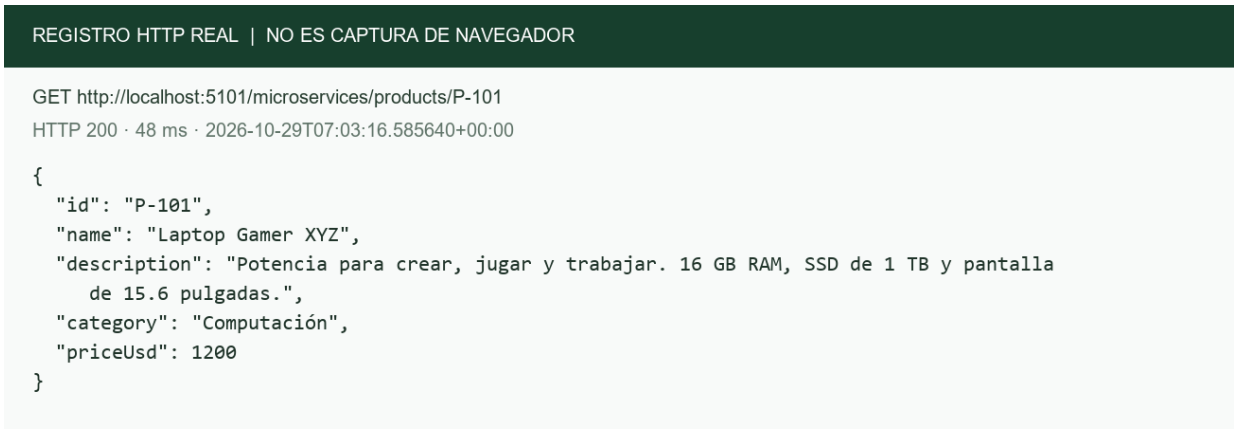


Figura 1. Catálogo propio. GET /microservices/products/P-101, HTTP 200. Se comprueban identidad del producto y precio base USD.



Figura 2. Inventario propio. GET /microservices/inventory/P-101, HTTP 200. Se confirman 15 unidades en el almacén Chihuahua A-12.

Capturas visuales pendientes: el acceso automatizado del navegador fue bloqueado por una configuración de permisos guardada. La autorización escrita del usuario no eliminó ese bloqueo. Deben añadirse las capturas del cliente REST o web después de habilitarlo; esta entrega no las declara realizadas.

5 Evidencias de divisas y consolidación

REGISTRO HTTP REAL | NO ES CAPTURA DE NAVEGADOR

GET https://api.frankfurter.dev/v2/rate/USD/MXN

HTTP 200 · 529 ms · 2026-10-29T07:03:17.164045+00:00


```
{
  "date": "2026-10-29",
  "base": "USD",
  "quote": "MXN",
  "rate": 18.6467
}
```


Figura 3. Consulta HTTPS real a Frankfurter v2. Se conservan la tasa USD/MXN y su fecha de referencia.

REGISTRO HTTP REAL | NO ES CAPTURA DE NAVEGADOR

GET http://localhost:5100/api/v1/products/P-101/summary

HTTP 200 · 120 ms · 2026-10-29T07:03:17.299127+00:00


```
{
  "product_id": "P-101",
  "title": "Laptop Gamer XYZ",
  "price_usd": 1200,
  "price_mxn": 22376.04,
  "stock_status": "IN_STOCK",
  "available_units": 15,
  "currency_conversion_source": "Frankfurter External API",
  "exchange_rate": 18.6467,
  "exchange_rate_date": "2026-10-29",
  "status": "SUCCESS"
}
```


Figura 4. Respuesta consolidada real. La figura muestra los campos principales; el JSON original contiene también la descripción y las tres observaciones de servicio.

En esta consulta, 1200.00 USD × 18.6467 MXN/USD = 22376.04 MXN. La conversión usa la tasa publicada para 2026-10-29; los valores cambiarán en ejecuciones futuras. El estado obtenido fue SUCCESS.

5 Pruebas de integración y reproducción

Se ejecutó dotnet build ECommerce.slnx con cero errores y cero advertencias. Después se ejecutó dotnet run --project tests/ECommerce.Tests --no-build: las 26 verificaciones terminaron satisfactoriamente, con código de salida cero. El archivo docs/test-results.txt conserva el registro completo.

Grupo	Casos verificados
Dominio y concurrencia	Conversión decimal, redondeo, rechazo de precio y stock negativos, barrera que exige iniciar las tres consultas y propagación de cancelación.
HTTP correcto y entradas	Catálogo e inventario independientes, contrato consolidado, alias de ruta, ID inválido 400, producto inexistente 404 y stock cero sin promesa de entrega.
Resiliencia	Divisas con 503, JSON malformado, tasa inválida, timeout, recuperación posterior, inventario caído, catálogo caído, traducción 503 a 502 y timeout crítico 504.
Presentación y contratos	Dashboard servido por el gateway, cabeceras de seguridad, inserción segura de texto y rechazo de un producto externo con identificador diferente.

Las pruebas HTTP levantan procesos reales y un servidor local de divisas controlado, para reproducir fallos sin depender de la disponibilidad de Internet. Esta simulación se limita al entorno de pruebas. Las cuatro consultas de las figuras anteriores sí se realizaron con los microservicios propios y Frankfurter real.

Cómo ejecutar y capturar la interfaz

```
dotnet build ECommerce.slnx
dotnet run --project tests/ECommerce.Tests --no-build
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/start.ps1
```

Abrir http://localhost:5100, consultar P-101 y desplegar Ver JSON. Luego probar P-103 y P-999. Para el cliente REST, importar o copiar docs/requests.http y capturar la URL y el cuerpo de las cuatro peticiones. Para verificar una respuesta parcial manual, ejecutar catálogo y gateway dejando inventario detenido.

Interfaz implementada

El dashboard incluye selector de productos, consulta y reintento, estado de stock, precios USD y MXN, unidades, almacén, entrega, fecha de tasa y tarjetas por dependencia con duración medida. Permite ver y copiar el JSON. El diseño se adapta mediante CSS a pantallas pequeñas. La verificación visual y la interacción en navegador permanecen pendientes del permiso señalado; la prueba HTTP confirmó que HTML y JavaScript se sirven correctamente.

6 Conclusiones y lecciones aprendidas

La solución muestra cómo integrar contratos que pertenecen a servicios diferentes sin trasladar esa coordinación al frontend. La API compuesta concentra el mapeo, la conversión de moneda y la clasificación de errores, mientras catálogo e inventario mantienen sus responsabilidades y procesos independientes.

Una lección central es distinguir la ausencia de un dato de un valor válido. Un inventario no disponible no equivale a cero unidades y una API de divisas caída no justifica inventar un precio local. El contrato parcial permite conservar información útil y comunicar su límite. También es necesario validar las respuestas externas, conservar su fecha y limitar el tiempo de espera.

La separación de dependencias facilitó probar concurrencia y resiliencia con dobles controlados, además de verificar rutas HTTP reales. El beneficio de reducir las consultas del cliente debe acompañarse de observabilidad y pruebas de carga si la solución evoluciona a producción: el gateway puede convertirse en un punto de concentración de fallos. Esta práctica no pretende acreditar esas pruebas adicionales.

7 Repositorio y flujo de trabajo

Repositorio de código fuente:

https://github.com/EdwinLedezma/practica-e-commerce

El repositorio se creó privado. Para evaluación externa se debe conceder acceso al docente o acordar su visibilidad. La solución se desarrolló en feature/composite-api-dashboard a partir de develop, conservando main como rama estable. Los cambios se proponen por PR de feature a develop y se promueven después mediante PR de develop a main. No se simulan aprobaciones humanas.

El README explica compilación, ejecución, variables, pruebas y límites. Los commits separan la API, el dashboard y las verificaciones; la evidencia de revisiones o integración solo debe darse por realizada cuando exista en GitHub. La entrega incluye un workflow de CI para compilar y ejecutar las verificaciones en cada push o PR.

Fuentes de información

Frankfurter. (s. f.). Exchange rate and currency data API. https://frankfurter.dev/

Microsoft. (s. f.-a). Gateway Aggregation pattern. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-aggregation

Microsoft. (s. f.-b). Common web application architectures. https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

Microsoft. (s. f.-c). IHttpClientFactory with .NET. https://learn.microsoft.com/en-us/dotnet/core/extensions/httpclient-factory

Ledezma Rivas, E. A. (2026). practica-e-commerce [Código fuente y registros de pruebas]. GitHub. https://github.com/EdwinLedezma/practica-e-commerce